

ПМ 1 — Рабочая учебная программа

Специальность 06130100 "Программное обеспечение (по видам)"



Квалификация: 4S06130105 Техник информационных систем

ЗНАНИЯ
ТЕХНОЛОГИИ
РАЗВИТИЕ
БУДУЩЕЕ

Сегодня
знания —
завтра
большие
возможности!



Больше
чем образование
— это возможности!

ПРОФЕССИОНАЛЫ
СОЗДАЮТ
БУДУЩЕЕ

1



Паспорт программы

ОСНОВНЫЕ ХАРАКТЕРИСТИКИ
УЧЕБНОГО МОДУЛЯ



Форма обучения:
очная



База:
**основное
среднее
образование**



Общий объём:
288 часов



Кредиты:
12



Семестры:
3-й и 4-й

2



О чём модуль



Модуль направлен на изучение постановки целей, задач и требований к программному обеспечению, разработки технических спецификаций, выбора методов разработки, подготовки макетов модулей ПО, работы с операционными системами, реализации численных алгоритмов и администрирования компьютерных сетей.

3



Распределение часов

СТРУКТУРА УЧЕБНОЙ НАГРУЗКИ

Теория



106 ч

Лабораторно-
практические



98 ч

СРСР



84 ч

Всего



288 ч

По семестрам:



3 семестр
246 ч



4 семестр
42 ч



Учебная практика
192 часа



288 часов —
комплексная подготовка
для профессионального
будущего!

5



Итог подготовки студента

К КАКИМ КОМПЕТЕНЦИЯМ ПРИВОДИТ ИЗУЧЕНИЕ МОДУЛЯ



анализировать
предметную область



составлять
требования
и спецификации



выбирать
стратегию
разработки



создавать макеты
и реализовывать
алгоритмы



работать
с ОС и сетями

4



Результаты обучения

Что студент будет знать, уметь и владеть

1

РО 1.1.

Цели, задачи
и требования к ПО



40 часов

- понятие и классификация ПО
- жизненный цикл и этапы разработки
- анализ предметной области и требования

2

РО 1.2.

Технические
спецификации



40 часов

- назначение спецификации
- описание компонентов и данных
- оформление техдокументации

3

РО 1.3.

Методы
разработки ПО



40 часов

- Waterfall и итерационная модель
- Agile, Scrum, Kanban
- сравнение и выбор стратегии

4

РО 1.4.

Макеты модулей ПО



42 часа

- модульная структура ПО
- UI/UX и прототипирование
- создание макетов экранных форм

5

РО 1.5.

ОС и прикладные
программы



42 часа

- функции ОС и файловая система
- установка и настройка программ
- безопасность работы в ОС

6

РО 1.6.

Численные алгоритмы



42 часа

- свойства и запись алгоритмов
- линейные, ветвящиеся и циклические алгоритмы
- реализация и тестирование

7

РО 1.7.

Компьютерные сети



42 часа

- виды и топологии сетей
- IP-адресация и локальная сеть
- доступ к ресурсам и защита

Σ

7 результатов обучения — комплексная подготовка будущего разработчика ПО

хекслет
колледж

ПМ 1

Разработка программного обеспечения

ТЕМА ЗАНЯТИЯ

Понятие программного обеспечения, его назначение и классификация



Изучаем



Разбираем



Применяем



Цель занятия

Что должно измениться в понимании студента после первого занятия



Хекслет
колледж



Сформировать профессиональное представление
о **программном обеспечении**, его роли в компьютерной системе,
назначении и основных классах.

01 Понимать

- разницу между Hardware и Software
- почему ПО является частью вычислительной системы
- почему программа не обязана иметь графический интерфейс



02 Различать

- системное ПО
- прикладное ПО
- инструментальное ПО
- данные и программные средства работы с ними



03 Объяснять

- назначение конкретной программы
- роль ПО в архитектуре системы
- почему разработчик отвечает не только за написание кода



Хекслет
колледж

Кейс: Mars Climate Orbiter

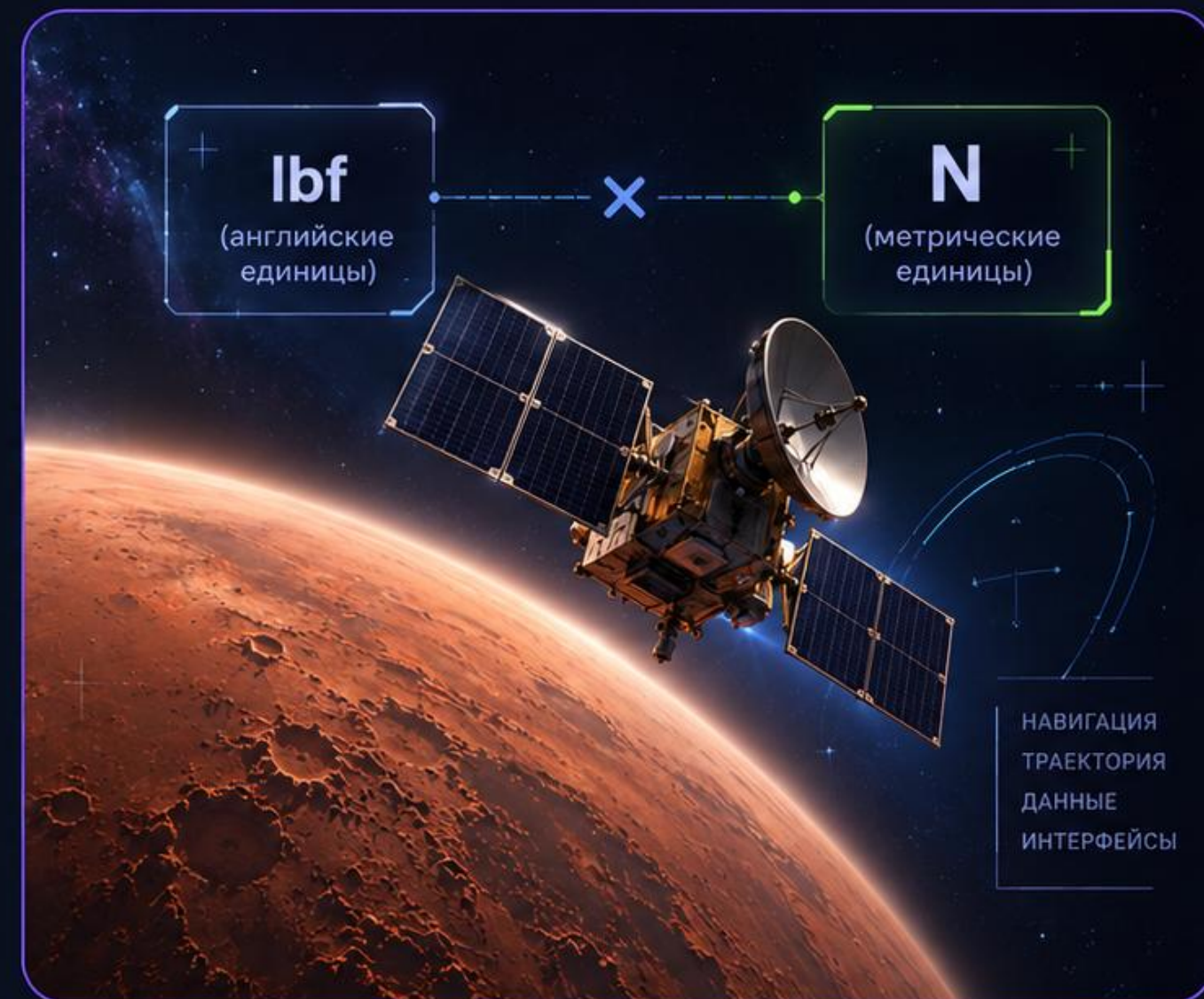


хекслет
колледж

Программная система может ошибиться не только «на экране».

1999 год. NASA потеряло аппарат Mars Climate Orbiter при подлёте к Марсу.

- Расследование установило несогласованность единиц измерения: один наземный программный компонент формировал данные в английских единицах, а интерфейс ожидал метрические.
- Ошибка повлияла на навигационные расчёты и фактическую траекторию аппарата.
- Этот пример показывает: качество программной системы зависит не только от синтаксически правильного кода, но и от корректных интерфейсов, требований, данных и взаимодействия компонентов.
- Инженерный вывод: «компонент работает» ещё не означает, что правильно работает вся система.



Вопрос аудитории: где именно здесь находится «ошибка ПО» — в одной строке кода или в организации взаимодействия компонентов?



Компьютерная система: Hardware + Software + Data



хекслет
колледж

Полноценный компьютер — это не только физическое устройство

01 Hardware — аппаратное обеспечение

- Физические компоненты, которые выполняют вычисления, хранят и передают данные.
- Примеры: CPU, RAM, SSD, GPU, сетевой адаптер, монитор, клавиатура, датчики.



02 Software — программное обеспечение

- Программы и программные компоненты, которые задают логику работы вычислительной системы.
- ПО определяет, какие действия система должна выполнять и как обрабатывать данные.



03 Data — данные

- Информация, которую система получает, хранит, изменяет и передаёт.
- Примеры: сообщения, фотографии, записи БД, координаты, логины, файлы, измерения датчиков.



Hardware выполняет. **Software** определяет, что выполнять. **Data** — то, с чем система работает.



Контрольная точка 1



хекслет
колледж

Hardware + Software + Data



Word Cloud



Отвечаем с телефона



30 секунд



1 вариант ответа



В смартфоне хранится фотография, которую пользователь открывает в приложении «Галерея». Что в этой ситуации относится к Data?



A



камера смартфона

B



приложение «Галерея»

C



фотография

D



операционная
система Android



После голосования **обсудим**, почему остальные варианты не относятся к данным.



Что такое программное обеспечение?



хекслет
колледж

От бытового понимания к профессиональному



Программное обеспечение — совокупность программ и связанных программных компонентов, предназначенных для выполнения определённых функций вычислительной системы.

Важно: программное обеспечение — это не только «приложение, которое запускает пользователь». В современной системе ПО может включать клиентскую часть, серверные процессы, системные службы, библиотеки, драйверы и другие взаимодействующие компоненты.

01 Программа

- Набор инструкций, которые может выполнить вычислительная система.
- Даже несколько строк кода уже могут образовывать программу.



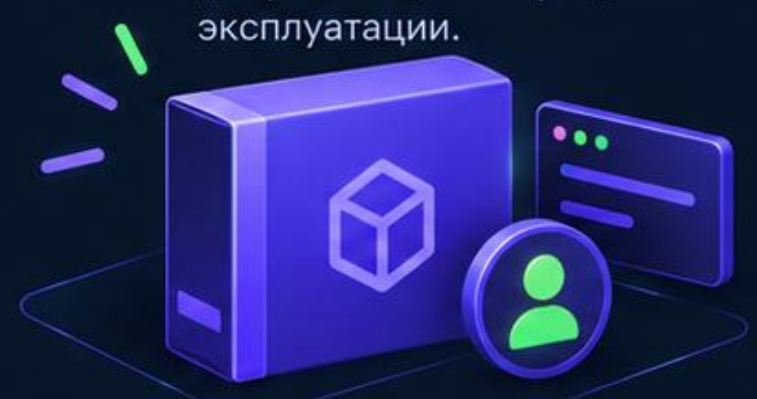
02 Программный компонент

- Часть более крупной программной системы, выполняющая определённую функцию.
- Компоненты взаимодействуют через данные и интерфейсы.



03 Программный продукт

- Законченное решение, предназначенное для пользователя или организации.
- Включает не только код, но и настройки, данные, документацию и процессы эксплуатации.



Разработчик не «просто пишет код»

Код — лишь одна стадия превращения проблемы в программный продукт

хекслет
колледж



Хороший разработчик умеет переводить реальную проблему
в **точные правила, данные, алгоритмы и проверяемое поведение программы.**

Программа не обязана **иметь окно**

Графический интерфейс — только один из способов взаимодействия



хекслет
колледж



Фоновые процессы

Работают в фоне и обеспечивают стабильную работу системы



системные службы



планировщики задач



службы синхронизации



мониторинг состояния системы



Серверное ПО

Работает на серверах и обрабатывает запросы



веб-серверы



серверы приложений



СУБД



API-сервисы



очереди сообщений



Встроенное ПО

Работает внутри устройств и управляет их функциями



программы микроконтроллеров



ПО маршрутизаторов



электроника автомобиля



контроллеры бытовой техники



Пример фоновой логики:

```
while True:  
    check_temperature(); send_metrics(); sleep(60)
```


Классификация ПО по назначению

Классификация помогает понять роль программы в системе



хекслет
колледж

01

Системное ПО

Обеспечивает работу всей вычислительной системы



Управляет ресурсами и устройствами



Создаёт среду для запуска других программ



Обеспечивает базовые функции системы

Примеры:

Windows, Linux, Android, драйверы.



02

Прикладное ПО

Решает конкретные задачи пользователя или организации



Работает с документами, сообщениями, графикой, навигацией, финансами и т. д.



Ориентировано на потребности пользователя



Упрощает выполнение повседневных задач

Примеры:

Telegram, Photoshop, 2GIS, игры, CRM.



03

Инструментальное ПО

Используется при создании, тестировании и сопровождении других программ



Обеспечивает работу разработчика



Предоставляет инструменты для разработки, отладки и тестирования



Включает компиляторы, интерпретаторы, среды разработки

Примеры:

VS Code, PyCharm, Git, компиляторы, отладчики.



Ключевой критерий классификации — **основное назначение продукта** в рассматриваемом контексте.

Системное программное обеспечение

Кто связывает пользовательские программы с аппаратными ресурсами?



хекслет
колледж



Управляет ресурсами системы

Операционная система контролирует процессы, память, файлы, пользователей и взаимодействие с устройствами.



Работает через драйверы

Драйвер — это специализированное ПО, которое позволяет операционной системе работать с конкретным устройством или классом устройств.



Предоставляет базовые сервисы

Системные службы выполняют фоновые функции: сеть, печать, обновления, безопасность, журналирование и т. д.



Помогает обслуживать и диагностировать

Системные утилиты упрощают настройку, обслуживание и диагностику компьютера.



Системное ПО — **основа**, благодаря которой программы могут работать с оборудованием.



Игра
(программа)



Операционная
система



Драйвер



GPU
(устройство)

Пример: игра не обязана напрямую знать аппаратные детали каждой видеокарты. Большую часть взаимодействия обеспечивают системные уровни и драйверы.

Примеры



Windows / Linux / Android
операционные системы



Драйвер NVIDIA или AMD
обеспечивают работу видеокарты



Службы сети и печати
позволяют использовать сетевые ресурсы
и подключённые устройства

Прикладное программное обеспечение

Программы, которые решают задачи пользователя или бизнеса



хекслет
колледж



Коммуникации

Помогают общаться и совместно работать

- Telegram, Discord, Zoom
- Решают прикладную задачу конечного пользователя



Контент

Позволяют создавать и обрабатывать контент

- Photoshop, Blender, DaVinci Resolve
- Решают прикладную задачу конечного пользователя



Навигация

Помогают ориентироваться в пространстве

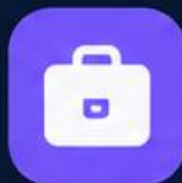
- 2GIS, Google Maps
- Решают прикладную задачу конечного пользователя



Финансы

Помогают управлять деньгами

- Мобильный банк, интернет-банкинг
- Решают прикладную задачу конечного пользователя



Бизнес

Автоматизируют бизнес-процессы

- CRM, ERP, документооборот
- Решают прикладную задачу конечного пользователя



Игры

Развлекают и объединяют людей

- Minecraft, Counter-Strike и др.
- Решают прикладную задачу конечного пользователя



Важно: прикладное ПО может быть настольным, мобильным, веб-приложением или частью крупной корпоративной системы.



Инструментальное ПО: рабочее место разработчика

Разработчик использует программы для создания других программ

хекслет
колледж



Редактор кода / IDE

Написание, навигация, рефакторинг и запуск проекта.



Компилятор или интерпретатор

Преобразование или выполнение исходного кода.



Отладчик

Пошаговое выполнение программы и анализ состояния.



Git

Система контроля версий исходного кода.



Средства тестирования

Автоматическая проверка поведения программы.



Инструменты сборки и развёртывания

Подготовка программы к запуску в рабочей среде.



Инструментальное ПО создаёт удобную и надёжную среду, в которой разработчик может эффективно **писать, проверять** и **выпускать код**.

Контрольная точка 2



хекслет
колледж

Классификация программного обеспечения



Word Cloud



Отвечаем с телефона



30 секунд



1 вариант ответа



Какой вариант правильно распределяет программы по классам ПО?



Windows — ?



Telegram — ?



VS Code — ?



A



прикладное



прикладное



системное

B



системное



прикладное



инструментальное

C



системное



инструментальное



прикладное

D



системное



системное



прикладное



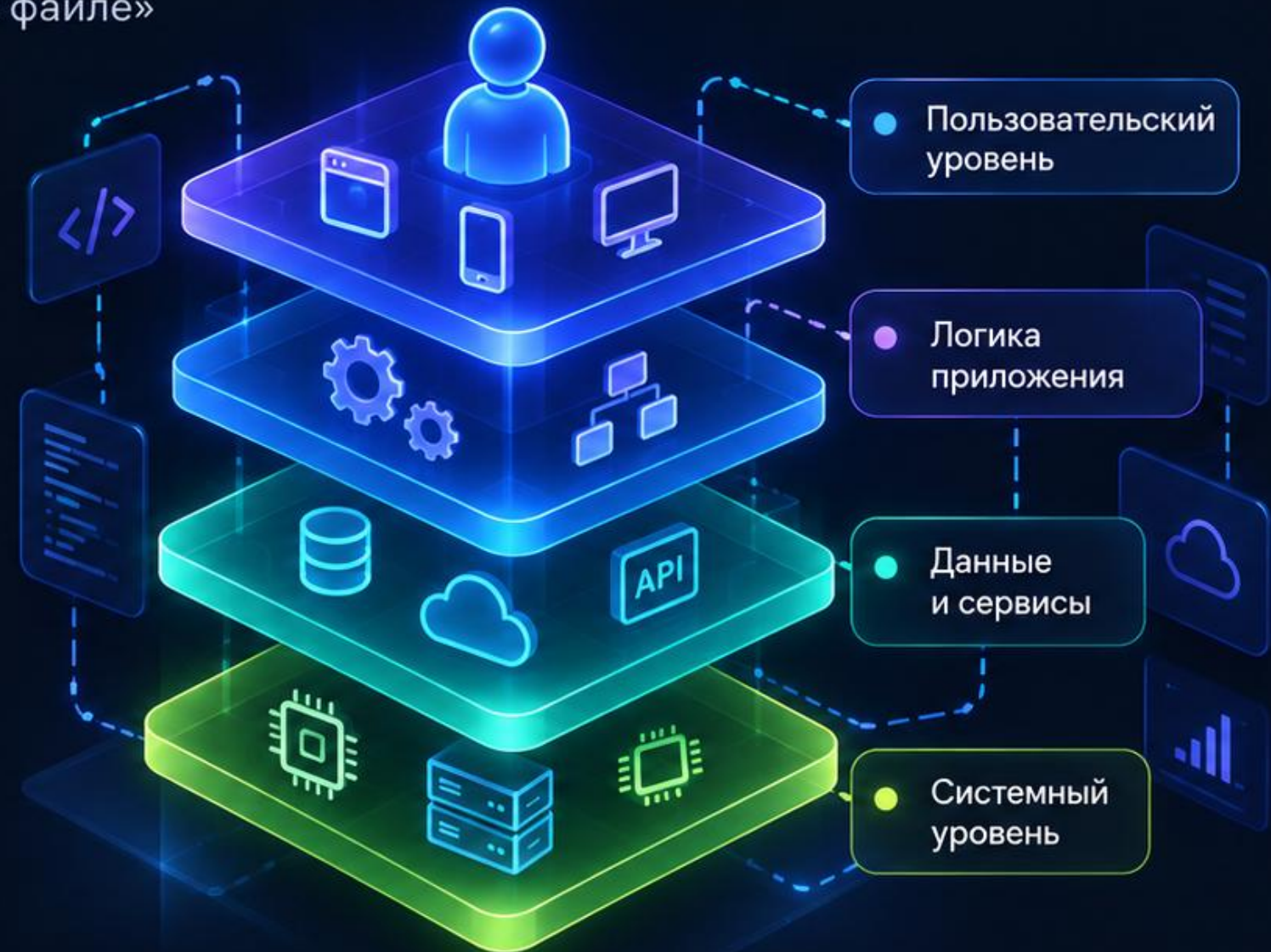
Ключевая идея: классификация зависит от основной функции продукта в системе.



Современная программная система состоит из уровней

хекслет
колледж

Один продукт редко является «одной программой в одном файле»



1. Пользовательский уровень

Интерфейс: веб-сайт, мобильное приложение, desktop-клиент.



2. Логика приложения

Правила обработки действий пользователя и бизнес-правила.



3. Данные и сервисы

Базы данных, API, файловые хранилища, внешние сервисы.



4. Системный уровень

Операционная система, сетевой стек, драйверы, аппаратные ресурсы.



Чем сложнее система, тем важнее точные интерфейсы между уровнями:
формат данных, единицы измерения, права доступа, ошибки и ожидаемое поведение.

Контрольная точка 3



хекслет
колледж

Программа не обязана иметь окно



Word Cloud



Отвечаем с телефона



30 секунд



1 вариант ответа



Какой пример лучше всего доказывает,
что программа может работать
без графического интерфейса?



A



графический редактор

B



фоновая служба
резервного копирования

C



браузер

D



текстовый редактор



Подсказка: интерфейс — это лишь один из способов взаимодействия с программой.



Кейс: Knight Capital – сотни миллионов за минуты

Программный сбой может иметь прямые финансовые последствия



хекслет
колледж



1 августа 2012 года торговая система Knight Capital начала отправлять на рынок **большое количество ошибочных заявок** после проблемного развёртывания программного обеспечения.



Компания сообщила примерно о **\$440 млн** торговых убытков — пример того, как сбой ПО может мгновенно повлиять на бизнес.



Проблема была связана не только с кодом, но и с процессом развёртывания, конфигурацией и работой старых компонентов на части серверов.



Кейс показывает важность контроля версий, единообразного развёртывания, тестирования, мониторинга и плана отката.



Инженерный вопрос: кто отвечает за качество ПО — только программист, написавший код, или вся команда и процесс разработки?



Как небольшой сбой превратился в огромные убытки

Один сбой.
Реальные последствия.



Обновление
Новая версия ПО



Развёртывание
На часть серверов



Старый компонент
Несовместимая конфигурация



Ошибочные заявки
Автоматическая отправка на рынок



Убытки
\$440 млн



Рынок получил
тысячи ошибочных
заявок за секунды

Не все случаи однозначны

Классификация зависит от основного назначения в конкретном контексте



Google Chrome

- Для обычного пользователя — прикладное ПО: просмотр сайтов и работа с веб-приложениями.
- Для веб-разработчика DevTools делает браузер ещё и инструментом разработки и диагностики.



Git

- Для большинства пользователей компьютера Git вообще не нужен.
- Для разработчика это инструмент управления версиями и совместной работы с исходным кодом.



PostgreSQL

- PostgreSQL — система управления базами данных, то есть программное обеспечение.
- База данных — организованная совокупность данных. Это не то же самое, что СУБД.



Правильный вопрос: какую основную функцию выполняет этот программный продукт в рассматриваемой системе?

Мини-задание: Software Detective

Определите роль каждого элемента в системе доставки



хекслет
колледж



- 01 Какие элементы относятся к **прикладному ПО**?
- 02 Что здесь является **системным ПО**, если сервер работает под Linux?
- 03 К какой категории вы отнесёте **VS Code** и **Git**, которыми пользуется разработчик backend?
- 04 Является ли PostgreSQL **базой данных** или **программой**? Сформулируйте различие.
- 05 Что произойдёт, если каждый компонент по отдельности работает правильно, но они используют **разные форматы данных**?



Short Answer



Главный вопрос: как все эти компоненты взаимодействуют, чтобы пользователь получил свой заказ?



Разбор мини-задания: Software Detective

Смотрим на систему как на единое целое и понимаем роль каждого элемента



хекслет
колледж



01 Какие элементы относятся к прикладному ПО?

 Прикладное ПО: мобильное приложение (клиентское ПО).

Обоснование:

- Прикладное ПО решает конкретные задачи пользователя (в нашем случае — заказ еды).
- Мобильное приложение предоставляет интерфейс и реализует пользовательскую логику.
- Остальные компоненты (ОС, сервер, СУБД) не являются прикладным ПО — они обеспечивают его работу.

02 Что здесь является системным ПО, если сервер работает под Linux?

 Системное ПО: операционная система Linux (на сервере).

Обоснование:

- Системное ПО управляет ресурсами компьютера, обеспечивает запуск программ и взаимодействие с оборудованием.
- Linux — это ОС, которая предоставляет среду для работы сервера, запуска backend-приложения и других служб (сеть, безопасность, управление процессами и т. д.).

03 К какой категории вы отнесёте VS Code и Git, которыми пользуется разработчик backend?

 Инструментальное ПО (средства разработки).

Обоснование:

- VS Code — это среда разработки (IDE), которая помогает писать, редактировать и отлаживать код.
- Git — система контроля версий, используемая для управления исходным кодом.
- Оба инструмента не решают задачи конечного пользователя, а помогают создавать и поддерживать программное обеспечение.

04 Является ли PostgreSQL базой данных или программой? Сформулируйте различие.

 PostgreSQL — это система управления базами данных (СУБД), то есть программа.

Обоснование:

- База данных — это сами данные, организованные по определённой структуре (таблицы, записи и т. д.).
- СУБД — это программное обеспечение, которое позволяет создавать, хранить, изменять и получать данные из базы.
- PostgreSQL — СУБД, а данные заказов, пользователей, ресторанов и т. д. — это база данных, которая хранится внутри PostgreSQL.

05 Что произойдёт, если каждый компонент по отдельности работает правильно, но они используют разные форматы данных?

 Система не будет работать корректно: компоненты не смогут понимать друг друга.

Обоснование:

- Даже если каждый компонент работает правильно, они должны обмениваться данными в согласованном формате (например, JSON, XML, бинарный протокол).
- Если форматы разные, возникнут ошибки при передаче и обработке данных (например, некорректный парсинг, потеря данных, сбой в логике).
- Поэтому в реальных системах важно использовать единые стандарты форматов данных и чётко описанные интерфейсы (API).



Главный вывод: программная система — это команда, где каждый компонент выполняет свою роль, а согласованные интерфейсы и форматы данных позволяют им работать вместе.



Итоги лекции

Что мы разобрали за 60 минут



хекслет
колледж



Компьютерная система как единое целое

01

Объединяет аппаратное обеспечение, программное обеспечение и данные.



Роль программного обеспечения

02

ПО задаёт логику работы системы и может существовать как пользовательское приложение, фоновый сервис, драйвер, серверный компонент или встроенная программа.



Классификация ПО по назначению

03

Удобно делить на системное, прикладное и инструментальное программное обеспечение.



Современные программные продукты

04

Часто состоят из множества уровней и компонентов, которые взаимодействуют через определённые интерфейсы.



Роль разработчика

05

Разработчик работает не только с кодом: он анализирует проблему, определяет требования, работает с данными, алгоритмами, архитектурой, тестированием и эксплуатацией.



Цена ошибок

06

Ошибки ПО могут иметь физические и финансовые последствия; поэтому качество системы зависит от инженерного процесса целиком.



Главный переход первого занятия:

от «я пишу код» к «я проектирую поведение программной системы».



ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

От идеи до реального мира

Программное обеспечение — это не просто приложения, которые мы используем каждый день. Это основа современного мира, которая управляет устройствами, хранит данные, решает задачи и делает технологии полезными.

Технологии
начинаются
с понимания



01 Компьютерная система Hardware + Software + Data

Полноценный компьютер — это гармоничное взаимодействие трёх компонентов.



Hardware
(аппаратное
обеспечение)

- Процессор
- Память (RAM)
- Накопители (SSD/HDD)
- Устройства ввода/вывода
- Сетевое оборудование



Software
(программное
обеспечение)

- Операционная система
- Прикладные программы
- Драйверы
- Системные службы
- Инструменты разработки



Data
(данные)

- Текст
- Изображения
- Видео
- Базы данных
- Результаты работы



Без программ Hardware — лишь «железо».
Без данных — нет смысла. Вместе они создают возможность
решать реальные задачи.

02 Что такое программное обеспечение?

Программное обеспечение (ПО) — совокупность программ и связанных компонентов, предназначенных для выполнения определённых функций вычислительной системы.



Программа
Набор инструкций, которые выполняет вычислительная система.

Компонент
Часть более крупной системы, выполняющая определённую функцию.

Продукт
Готовое решение для пользователя или организации.

03 Программа, компонент, продукт — в чём разница?



Программа

- Может быть небольшой (несколько строк).
- Решает конкретную задачу.
- Пример: скрипт на Python.



Программный компонент

- Входит в состав более крупной системы.
- Взаимодействует с другими компонентами.
- Пример: модуль авторизации.



Программный продукт

- Законченное решение.
 - Включает код, данные, настройки, документацию, процессы.
- Пример: Telegram, Photoshop, Windows.

04 Классификация ПО по назначению

Программное обеспечение делится на три основные категории.

Системное ПО



- Обеспечивает работу всей системы.
- Управляет ресурсами.
- Создаёт среду для других программ.

Примеры:
Windows, Linux, драйверы, системные службы.

Прикладное ПО



- Решает конкретные задачи пользователя.
- Работает с данными.
- Удобно и понятно для конечного пользователя.

Примеры:
Telegram, Google, Chrome, Photoshop, Excel, игры.

Инструментальное ПО



- Используется для создания других программ.
- Помогает писать, отлаживать и тестировать код.

Примеры:
VS Code, Git, компиляторы, отладчики.

05 ПО не обязано иметь окно

Многие программы работают в фоновом режиме и не имеют графического интерфейса.



Примеры таких программ:

- Серверные приложения
- Системные службы
- Скрипты и планировщики задач
- Встроенное ПО (микроконтроллеры)



Отсутствие окна не означает, что программа не работает.

06 Современная программная система состоит из уровней

Реальные продукты включают несколько уровней, которые взаимодействуют между собой.



Пользовательский уровень
(интерфейс)

Прикладной уровень
(бизнес-логика)

Уровень данных
(БД, API, файлы)

Системный уровень
(ОС, драйверы, оборудование)

08 Роль разработчика ПО



- Анализирует требования
- Проектирует решение
- Пишет код
- Работает с данными
- Тестирует и отлаживает
- Участвует в развёртывании и сопровождении

Разработчик — это не только человек, который пишет код, но и инженер, который создаёт надёжные и полезные решения.

09 Современные программные продукты в жизни и бизнесе



Коммуникации



Контент



Навигация



Финансы



Бизнес



Образование



Здравоохранение



Развлечения

10 Главные выводы

- ✓ Программное обеспечение — основа современных технологий.
- ✓ Компьютерная система = Hardware + Software + Data.
- ✓ ПО делится на системное, прикладное и инструментальное.
- ✓ Программа не обязана иметь графический интерфейс.
- ✓ Реальные системы состоят из множества уровней и компонентов.
- ✓ Качество ПО зависит не только от кода, но и от данных, интерфейсов, требований, тестирования и процессов.
- ✓ Разработчик отвечает за всё решение, а не только за написание кода.

Программное обеспечение меняет мир. Ты можешь быть частью этих изменений

Понимание
системы —
первый шаг
к созданию
больших
решений!

Контрольные вопросы

Ответьте полным предложением и приведите собственный пример



хекслет
колледж

01



Что такое программное обеспечение?

02



В чём принципиальное отличие Hardware от Software?

03



Почему мощный компьютер без программного обеспечения практически бесполезен обычному пользователю?

04



Назовите три основных класса ПО по назначению и объясните различия между ними.

05



Почему драйвер устройства относится к системному ПО?

06



Почему Visual Studio Code и Git относят к инструментам разработки?

07



Может ли программа работать без графического интерфейса? Приведите минимум два примера.

08



В чём разница между базой данных и СУБД?

09



Почему Google Chrome нельзя всегда однозначно характеризовать только одной ролью?

10



Какие выводы для разработчика можно сделать из историй Mars Climate Orbiter и Knight Capital?

11



Почему фраза «ошибка в одной строке кода» часто слишком упрощает причины сбоя сложной программной системы?

12



Какие этапы работы разработчика происходят до написания кода?



Думайте, анализируйте, приводите примеры из **реальной жизни** или из **изученного материала**!



Источники для инженерных кейсов

Для подтверждения фактов, использованных в лекции



хехслет
колледж



Mars Climate Orbiter



NASA — Mars Climate Orbiter — официальная страница миссии и материалы исследования.



Ключевой факт лекции: несогласованность английских и метрических единиц в программном интерфейсе наземных систем.



science.nasa.gov/mission/mars-climate-orbiter/



Официальный источник NASA о миссии Mars Climate Orbiter



Knight Capital



U.S. Securities and Exchange Commission и официальная отчётность Knight Capital Group.



Ключевой факт лекции: ошибочное развёртывание торгового ПО 1 августа 2012 года привело к массовой отправке заявок и убыткам порядка сотен миллионоов долларов.



sec.gov (Knight Capital Group, 2012)



Официальные документы SEC по инциденту Knight Capital



Используйте **надёжные источники**, чтобы опираться на проверенные факты

